TEC | Tecnológico de Costa Rica

Instituto Tecnológico de Costa Rica

Escuela de Computación

Compiladores e Intérpretes - IC5701

Proyecto #2

Análisis Semántico y Generación de Código Intermedio

Estudiantes:

Alice Arias Salazar | 2023104639

Heldyis Agüero Espinoza | 2023296812

Profesor:

Allan Rodriguez Davila

Grupo 60

Entrega: lunes 9 de noviembre 2026

II Semestre 2026

Manual de usuario

El proyecto se encuentra guardado en el repositorio de GitHub del equipo:

https://github.com/Alice-Arias/PY2-Analisis-Semantico-y-Generacion-Codigo-Intermedio-Compiladores

El programa recibe un archivo de código con extensión .cmm y realiza cuatro fases. Primero, separa el texto en palabras, números y símbolos análisis léxico. Después, revisa que estén ordenados correctamente análisis sintáctico. Luego, comprueba que el programa tenga sentido y que los tipos de datos se utilicen correctamente análisis semántico y si no encuentra errores, genera un archivo con una versión más sencilla del programa, llamada código intermedio.

El proyecto está desarrollado en Java y utiliza JFlex para el análisis léxico y Java CUP para los análisis sintáctico y semántico. Ambas librerías ya están incluidas en el repositorio, por lo que no es necesario descargarlas por separado.

1. Requisitos necesarios

Paso 1. Verificar los programas necesarios

Para usar el proyecto se necesita una computadora con Windows, Visual Studio Code y Java JDK 21. También se puede usar Git si se quiere copiar el proyecto con comandos, pero no es obligatorio: el proyecto también se puede descargar como un archivo ZIP. No hace falta instalar JFlex ni Java CUP. Las versiones que usa el proyecto ya están en la carpeta Programa\librerias.

2. Obtener el proyecto desde GitHub

Paso 2. Abrir el repositorio en el navegador

Primero se abre un navegador. En la barra de direcciones se escribe la dirección del repositorio y se presiona Enter. Se abrirá la página del repositorio, donde se ven las carpetas y los archivos del proyecto.

Paso 3. Descargar el proyecto como ZIP

Para una persona que no conoce Git, esta es la forma más fácil. En la página del repositorio se presiona el botón verde Code y, en el menú que aparece, se elige Download ZIP. El navegador descargará un archivo comprimido con una copia completa del proyecto, con un nombre parecido a PY2-Analisis-Semantico-y-Generacion-Codigo-Intermedio-Compiladores-main.zip.



Paso 4. Extraer el proyecto

El archivo se guarda normalmente en la carpeta Descargas. Se hace clic derecho sobre el ZIP, se elige Extraer todo… y se escoge dónde guardarlo. Después hay que entrar a la carpeta que realmente contiene el proyecto: la que tiene las carpetas Programa y Documentación y el archivo README.md. Si al abrirla solo aparece otra carpeta con el mismo nombre, se entra en ella.

3. Abrir el proyecto en Visual Studio Code

Paso 5. Abrir la carpeta del proyecto

Se abre Visual Studio Code y se elige File - Open Folder…. Se busca la carpeta del proyecto y se presiona Select Folder. No se debe crear un proyecto Java nuevo, porque el proyecto ya trae todos sus archivos.

4. Verificar que las librerías ya están incluidas

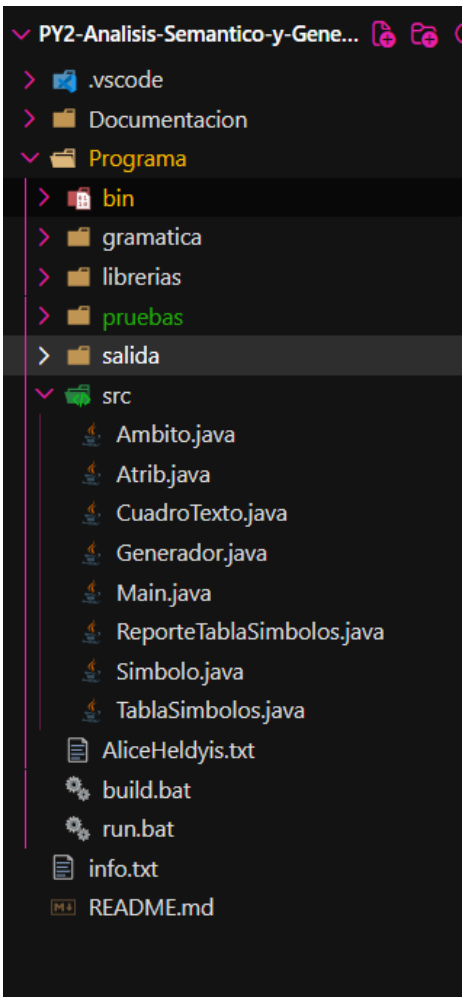
Paso 6. Comprobar la carpeta librerias

Dentro de Programa\librerias están JFlex 1.9.1 (jflex-full-1.9.1.jar) y Java CUP 11b (java-cup-11b.jar), así que no hay que descargarlas. No se les debe cambiar el nombre, ni moverlas, ni reemplazarlas por otras versiones: build.bat y run.bat las buscan en esa carpeta.

6. Conocer la estructura del proyecto

Paso 8. Identificar las carpetas y archivos principales

El proyecto separa lo que se escribe a mano de lo que se genera solo. Así se ve la carpeta del proyecto:



Estos son los archivos y carpetas principales:

- **build.bat y run.bat:** compilan y ejecutan el proyecto con un solo comando. Están directamente en la carpeta Programa.
- **gramatica\Lexer.flex:** tiene las reglas para separar el código en sus partes palabras, números, símbolos. JFlex lo usa para crear el analizador léxico.
- **gramatica\Parser.cup:** tiene las reglas del lenguaje y las revisiones de sentido tipos, nombres, funciones. Java CUP lo usa para crear el analizador sintáctico y semántico. También le pide al generador que escriba el código intermedio.
- **src\Main.java:** es el programa principal. Coordina las cuatro fases y muestra los resultados en pantalla.
- **src\TablaSimbolos.java, Simbolo.java, Ambito.java, ReporteTablaSimbolos.java y CuadroTexto.java:** forman la tabla de símbolos, una clase por archivo. Guardan las variables, los arreglos y las funciones del programa, organizados por ámbitos.

- **src\Atrib.java:** guarda los datos de cada valor mientras se analiza: su tipo y dónde queda su resultado.
- **src\Generador.java:** escribe el código intermedio.
- **pruebas:** tiene los archivos .cmm para probar el programa, separados en correctas y con_errores.
- **librerias:** tiene JFlex y Java CUP.
- **salida:** guarda todo lo que genera el programa: los tokens, la tabla de símbolos y el código intermedio.
- **bin:** tiene los archivos que se generan al compilar.

7. Importante: no modificar bin

Paso 9. Entender para qué sirven las carpetas que se llenan solas

Los archivos de bin (Lexer.java, parser.java, sym.java y los .class) los crea la compilación: no se deben editar, mover ni borrar, y se vuelven a crear solos cada vez que se compila. En cambio, la carpeta salida guarda los resultados de cada ejecución; esos archivos se pueden abrir y leer, y si alguna carpeta no existe, el programa la crea sola.

8. Compilar el proyecto

Paso 10. Entrar a la carpeta Programa

Los archivos que compilan y ejecutan el proyecto están directamente en la carpeta Programa. En la terminal se escribe:

```
cd .\Programa
```

La terminal debe terminar mostrando una ruta parecida a:

```
PS C:\Users\NombreUsuario\Desktop\PY2-...-Compiladores\Programa>
```

Paso 11. Ejecutar build.bat

Desde la carpeta Programa se escribe:

```
.\build.bat
```

y se presiona Enter. Este archivo hace todo el trabajo de compilar. Primero busca las librerías que están en Programa\librerias. Después usa Java CUP para crear el analizador del lenguaje a partir de gramatica\Parser.cup, usa JFlex para crear el separador de palabras a partir de gramatica\Lexer.flex y, por último, usa javac para compilar los archivos de Java. Al terminar debe aparecer el mensaje COMPILACION COMPLETADA, y Java CUP debe indicar 0 errors and 0 warnings y 0 conflicts detected.

9. Ejecutar el programa

Paso 12. Ejecutar run.bat

Cuando la compilación terminó bien, se sigue en la misma carpeta (Programa) y se escribe:

.\run.bat

y se presiona Enter. Este archivo muestra la lista de archivos de prueba disponibles, separados en correctas y con errores, y pide el nombre del archivo .cmm que se quiere analizar.

```
PS C:\Users\arias\...\PY2-Analisis-Semantico-y-Generacion-Codigo-Intermedio-Compiladores\Programa> .\run.bat

  Pruebas correctas   (pruebas\correctas):
    prueba_generador.cmm
    prueba_tac.cmm
    pruebasX.cmm

  Pruebas con errores (pruebas\con_errores):
    ejemplo1.cmm
    ejemplo2.cmm
    prueba_correccion_global.cmm
    prueba_recuperacion.cmm
    prueba_sin_principal.cmm

  Nombre del archivo .cmm: pruebasX
```

Paso 13. Seleccionar un archivo .cmm

Se escribe el nombre del archivo que se quiere probar y se presiona Enter. No hace falta escribir la carpeta ni la extensión .cmm: run.bat busca el archivo en pruebas\correctas y en pruebas\con_errores, y agrega la extensión si falta.

10. Escribir un archivo .cmm

Paso 14. Conocer los símbolos del lenguaje

El lenguaje usa algunos símbolos especiales en lugar de los que se usan en otros lenguajes, como las llaves o el punto y coma. Estos son los más importantes:

Símbolo	Para qué sirve
¿: y :?	Abren y cierran un bloque (donde en otros lenguajes irían llaves)
ϵ: y :ɜ	Abren y cierran un paréntesis
ʃ: y :ɻ	Abren y cierran un corchete (para la posición de un arreglo)
»	Marca el final de una instrucción (como el punto y coma)
Ⱶ	Asignación: guarda un valor en una variable
ʌ θ Σ	Y lógico, O lógico y negación
|	Comienza un comentario de una línea

Paso 15. Ver un programa de ejemplo

Un programa tiene una o más funciones y exactamente una función llamada principal. Este es un ejemplo corto:

```
int doble ϵ: int n :ɜ ¿:
   return n + n»
:?

void principal ϵ: :ɜ ¿:
   val int i Ⱶ 0»
   while ϵ: ʃ:i:ɻ < ʃ:3:ɻ :ɜ ¿:
      i Ⱶ i + 1»
   :?
   write ϵ: doble ϵ: i :ɜ :ɜ»
:?
```

La función doble recibe un número y devuelve su doble. La función principal crea la variable i con el valor 0, la aumenta de uno en uno hasta llegar a 3 y escribe el doble del resultado.

Paso 16. Recordar las reglas más importantes

- Las variables se crean con la palabra val y siempre llevan su tipo: int, float, bool, char o string.
- **Los tipos no se mezclan:** un valor decimal no puede guardarse en una variable entera.
- Una función debe escribirse antes de usarla.
- Las condiciones de if, elif, while y for deben ser de tipo bool, es decir, verdadero o falso.
- Los arreglos son de tipo int o float, y la primera posición es la número 0.

Todas las reglas, con ejemplos, están en la sección Reglas semánticas del analizador al final de este documento.

11. Entender lo que muestra el programa

Paso 17. Análisis léxico

Después de seleccionar el archivo .cmm, el programa comienza a leerlo e identificar sus elementos, como palabras reservadas, nombres, números, textos, operadores y símbolos. Cada elemento reconocido se llama token. Si encuentra algo que no pertenece al lenguaje, como el carácter @, muestra un error indicando la línea y la columna, y continúa con el análisis.

Paso 18. Generación del archivo de tokens

Durante el análisis se crea un archivo de tokens por cada programa analizado:

Programa\salida\tokens\<nombre>_tokens.txt

Ahí se guardan todas las partes reconocidas, con su línea, su columna y lo que decía el texto. Como cada archivo lleva el nombre del programa, los resultados de una prueba no borran los de otra.

Paso 19. Análisis sintáctico

El programa revisa que las instrucciones sigan el orden correcto. Si encuentra errores, muestra la línea, la ubicación, lo encontrado y lo esperado. También indica si logró corregirlos automáticamente.

Paso 20. Análisis semántico

El programa verifica que el código tenga sentido, como el uso correcto de variables, operaciones, funciones y condiciones. Los errores se muestran con su línea, columna, tipo y explicación. Al final, presenta un resumen de los errores encontrados.

Paso 21. Código intermedio

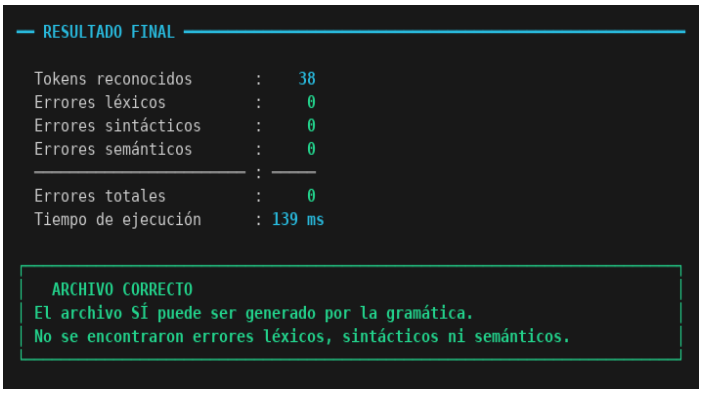
Si no hay errores, el programa genera el código intermedio, donde cada línea representa una operación sencilla. Se guarda en salida\codigo_intermedio con el mismo nombre del archivo y la extensión .tac. Si hay errores, no se genera.

Para leerlo hay que conocer cuatro cosas:

- **Temporales con su tipo:** cada resultado parcial se guarda en un temporal cuyo nombre dice qué tipo de dato guarda: int1, float2, bool3, char4 o string5. El número nunca se repite.
- **Etiquetas con $:** las funciones se marcan con tipo$nombre (por ejemplo int$suma) y terminan en tipo$nombre$final. Los if, while y for usan etiquetas como principal$while$6$condicion. Se usa $ porque ningún nombre del programa lo puede tener, así una etiqueta nunca se confunde con una variable.
- **Declaraciones:** una variable sin valor inicial aparece como declaracion# int x, y una con valor como declaracionPrevia# int x, justo antes del código que calcula su valor. Los parámetros de una función aparecen como param_int a.
- **Llamadas:** cada dato que recibe una función se envía con param, y la llamada se escribe como call tipo$nombre, cantidad de datos.

Paso 22. Resultado final

Al final se muestra un resumen con la cantidad de errores de cada fase y el tiempo que tardó el programa. También se indica si el archivo SÍ puede o NO puede ser generado por la gramática, es decir, si el programa está bien escrito y tiene sentido. Este resumen aparece antes de las preguntas, así el tiempo no cuenta lo que la persona tarda en responder.



Paso 23. Responder las preguntas finales

Al terminar, el programa hace dos preguntas. Se responde s para decir que sí o n para decir que no:

- **¿Desea ver el código intermedio?** Solo aparece si el código se generó. Si se responde s, se muestra el código con cada línea numerada y con colores: amarillo para las funciones, cian para las etiquetas y los saltos, verde para las declaraciones y rojo para los operadores.
- **¿Desea ver la tabla de símbolos?** Si se responde s, se muestra el árbol de ámbitos y la tabla con todas las variables, arreglos y funciones.

La tabla de símbolos siempre se guarda en un archivo, aunque se responda n. El archivo tiene exactamente el mismo contenido que se ve en la consola, sin los colores.

Paso 24. Saber qué archivos crea el programa

Archivo	Dónde queda	Qué contiene
<nombre>_tokens.txt	Programa\salida\tokens	Las partes reconocidas del programa
<nombre>_tabla_simbolos.txt	Programa\salida\tabla_simbolos	El árbol de ámbitos y la tabla de símbolos
<nombre>.tac	Programa\salida\codigo_intermedio	El código intermedio (solo si no hubo errores)

Pruebas de funcionalidad

Prueba 1. Compilación del proyecto

Se ejecuta build.bat para comprobar que el proyecto compile correctamente. En la imagen se observa que Java CUP generó el analizador sin errores ni advertencias y sin conflictos en la gramática, que JFlex generó el analizador léxico y que al final aparece el mensaje COMPILACION COMPLETADA.

```
PS C:\Users\arias\...\PY2-Analisis-Semantico-y-Generacion-Codigo-Intermedio-Compiladores\Programa> .\build.bat

============================================================
               COMPILACION DEL PROYECTO
============================================================

Proyecto : C:\Users\arias\...\PY2-Analisis-Semantico-y-Generacion-Codigo-Intermedio-Compiladores\Programa
JFlex    : ...\Programa\librerias\jflex-full-1.9.1.jar
CUP      : ...\Programa\librerias\java-cup-11b.jar
Runtime  : ...\Programa\librerias\java-cup-11b.jar
Bin      : ...\Programa\bin

Limpiando archivos generados anteriores...
Limpieza de bin completada.

Generando Parser con CUP...
------- CUP v0.11b 20150326 (SVN rev 63) Parser Generation Summary -------
  0 errors and 0 warnings
  53 terminals, 109 non-terminals, and 241 productions declared,
  producing 446 unique parse states.
  0 terminals declared but not used.
  0 non-terminals declared but not used.
  0 productions never reduced.
  0 conflicts detected (0 expected).
  Code written to "parser.java", and "sym.java".
-------------------------------------------------- (CUP v0.11b 20150326 (SVN rev 63))
CUP generado correctamente.

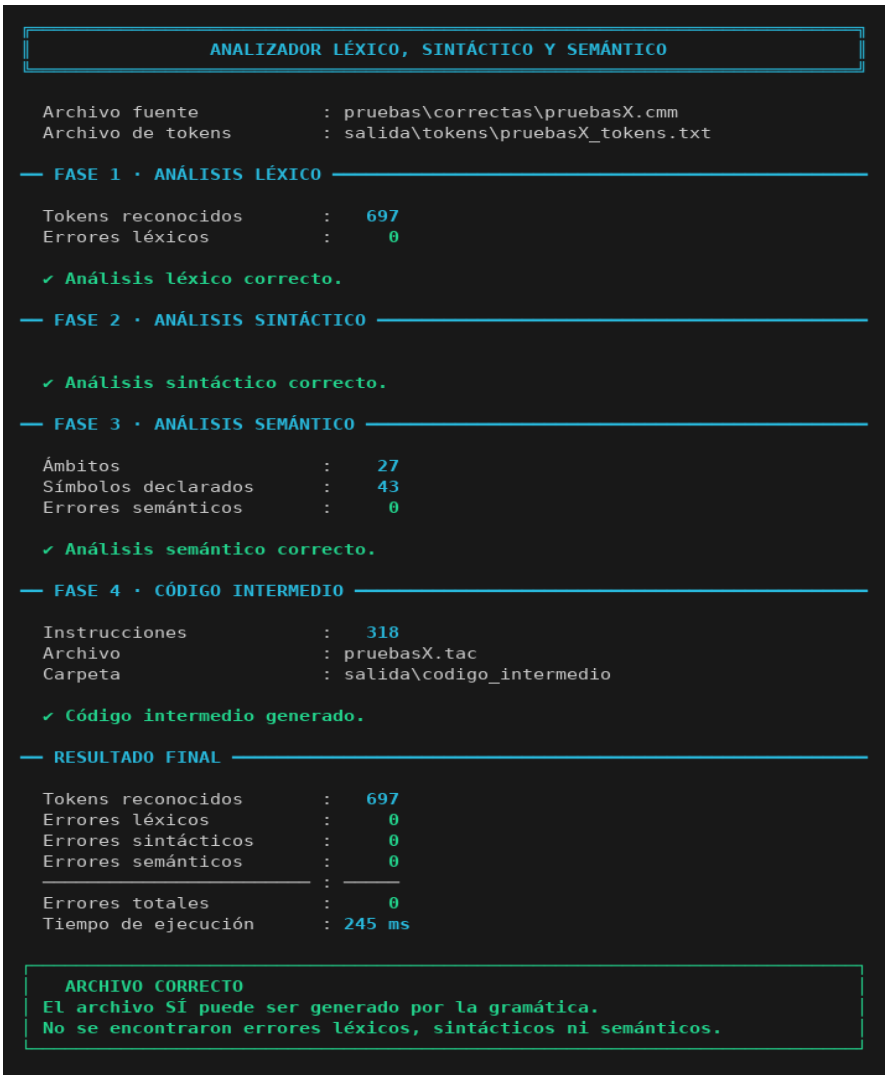
Generando Lexer con JFlex...
Reading "Lexer.flex"
Constructing NFA : 393 states in NFA
Converting NFA to DFA :
.....................................................................
165 states before minimization, 146 states in minimized DFA
Writing code to "C:\Users\arias\...\PY2-Analisis-Semantico-y-Generacion-Codigo-Intermedio-Compiladores\Programa\bin\Lexer.java"
JFlex generado correctamente.

Compilando archivos Java...

============================================================
            COMPILACION COMPLETADA
============================================================
```

Prueba 2. Un programa correcto

Se ejecuta pruebasX.cmm, un programa que utiliza variables globales, arreglos, cinco funciones, condiciones, ciclos y todas las operaciones del lenguaje. Todas las fases finalizan correctamente: se reconocen 697 tokens, se crean 27 ámbitos con 43 símbolos y se generan 318 instrucciones de código intermedio. No se presentan errores y el archivo puede ser generado por la gramática..



Ejecución de pruebasX.cmm, sin errores.

Prueba 3. Código intermedio de un programa corto

Se analiza el archivo prueba_tac.cmm, que declara variables y hace operaciones:

```
void principal ϵ: :ɜ ¿:
   val int a ⊦ 1»
   val int b ⊦ 2»
   val int c ⊦ 0»
   c ⊦ a + b * 2»
   c ⊦ a mod b»
:?
```

En la imagen se muestra el código intermedio generado. Cada variable con valor inicial empieza con su declaracionPrevia#. Los números y las variables se copian primero a un temporal con su tipo (int1 = 1, int6 = a), y cada operación se divide en pasos pequeños: a + b * 2 se calcula en dos instrucciones. Al final están el return y la etiqueta void$principal$final, que marca el fin de la función principal.

En este lenguaje la suma se resuelve antes que la multiplicación (el orden de importancia es pot, + -, * / //, mod), por eso a + b * 2 se calcula como (a + b) * 2.



Código intermedio de prueba_tac.cmm.

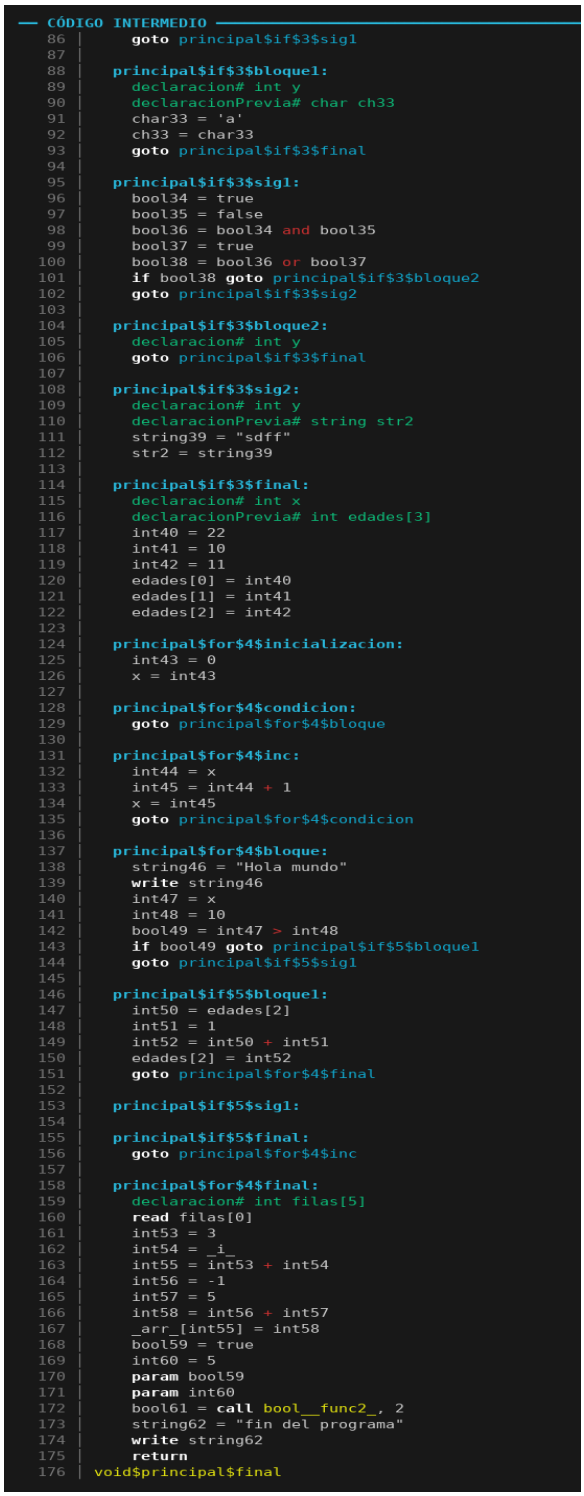
Prueba 4. Código intermedio de un programa completo

Se ejecuta prueba_generador.cmm, que contiene variables globales, arreglos, funciones, ciclos, condiciones y llamadas. Se generan 126 instrucciones de código intermedio.

En la primera parte aparecen las variables globales, las funciones y los ciclos while, representados mediante etiquetas. También se muestra cómo funciona break.

En la segunda parte se observan las condiciones if, elif y else, el ciclo for, el acceso a arreglos y las llamadas a funciones mediante param y call.

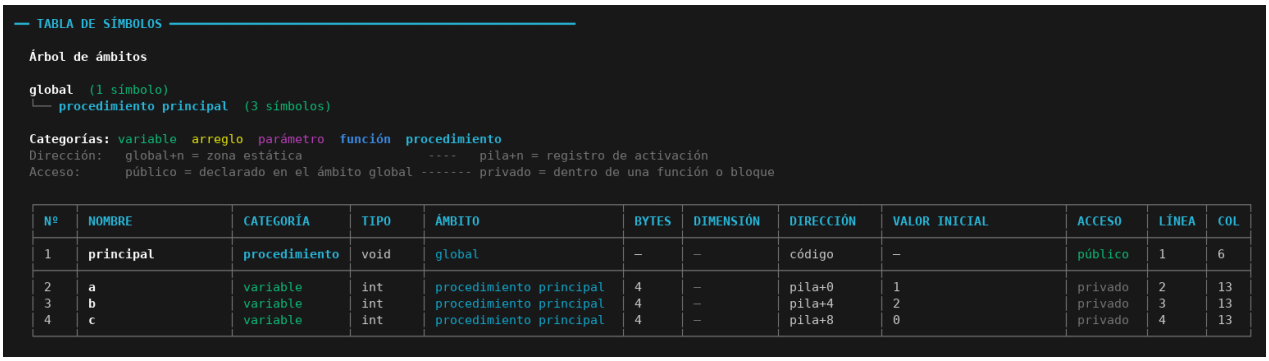




Prueba 5. Tabla de símbolos

Al finalizar el análisis, el programa pregunta si se desea ver la tabla de símbolos. Primero muestra el árbol de ámbitos y luego una tabla con las variables y funciones, indicando su nombre, categoría, tipo, ámbito, tamaño, dimensión, dirección, valor inicial, visibilidad y ubicación donde se declararon.

Tabla de símbolos de prueba_tac.cmm.



— TABLA DE SÍMBOLOS —

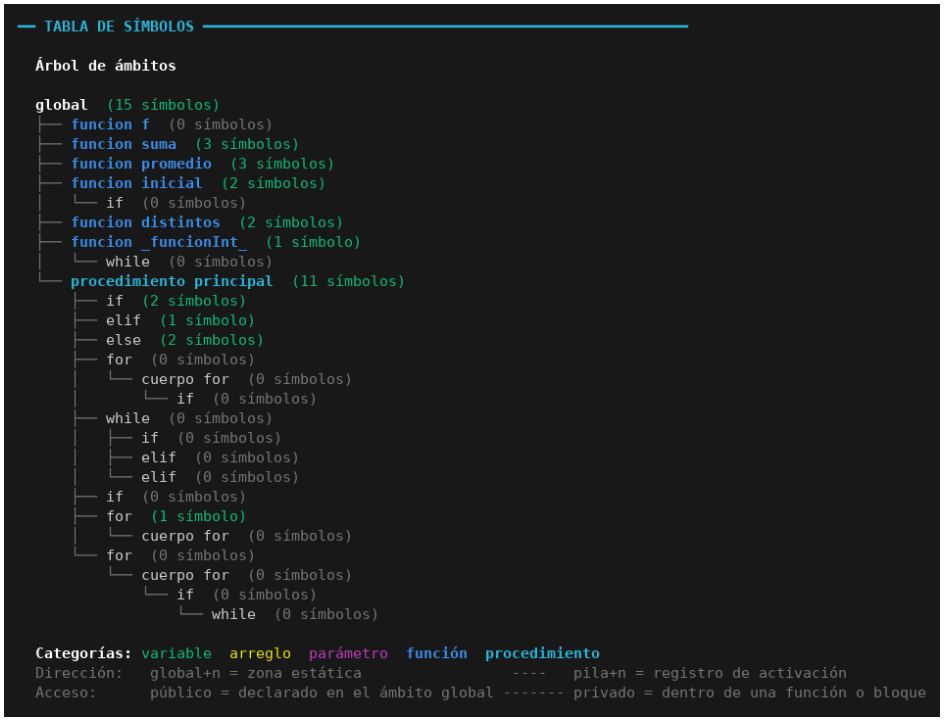
Árbol de ámbitos

global (1 símbolo)
└── procedimiento principal (3 símbolos)

Categorías: variable arreglo parámetro función procedimiento
Dirección: global+n = zona estática ---- pila+n = registro de activación
Acceso: público = declarado en el ámbito global ------- privado = dentro de una función o bloque

Nº	NOMBRE	CATEGORÍA	TIPO	ÁMBITO	BYTES	DIMENSIÓN	DIRECCIÓN	VALOR INICIAL	ACCESO	LÍNEA	COL
1	principal	procedimiento	void	global	–	–	código	–	público	1	6
2	a	variable	int	procedimiento principal	4	–	pila+0	1	privado	2	13
3	b	variable	int	procedimiento principal	4	–	pila+4	2	privado	3	13
4	c	variable	int	procedimiento principal	4	–	pila+8	0	privado	4	13

En un programa más grande, el árbol muestra cómo los bloques están unos dentro de otros. Los colores son los mismos de la leyenda de categorías: las funciones en azul, el procedimiento principal en cian y la cantidad de símbolos en verde cuando el bloque tiene alguno.



— TABLA DE SÍMBOLOS —

Árbol de ámbitos

global (15 símbolos)
├── funcion f (0 símbolos)
├── funcion suma (3 símbolos)
├── funcion promedio (3 símbolos)
├── funcion inicial (2 símbolos)
│ └── if (0 símbolos)
├── funcion distintos (2 símbolos)
├── funcion _funcionInt_ (1 símbolo)
│ └── while (0 símbolos)
└── procedimiento principal (11 símbolos)
├── if (2 símbolos)
├── elif (1 símbolo)
├── else (2 símbolos)
├── for (0 símbolos)
│ └── cuerpo for (0 símbolos)
│ └── if (0 símbolos)
├── while (0 símbolos)
│ ├── if (0 símbolos)
│ ├── elif (0 símbolos)
│ └── elif (0 símbolos)
├── if (0 símbolos)
├── for (1 símbolo)
│ └── cuerpo for (0 símbolos)
└── for (0 símbolos)
└── cuerpo for (0 símbolos)
└── if (0 símbolos)
└── while (0 símbolos)

Categorías: variable arreglo parámetro función procedimiento
Dirección: global+n = zona estática ---- pila+n = registro de activación
Acceso: público = declarado en el ámbito global ------- privado = dentro de una función o bloque

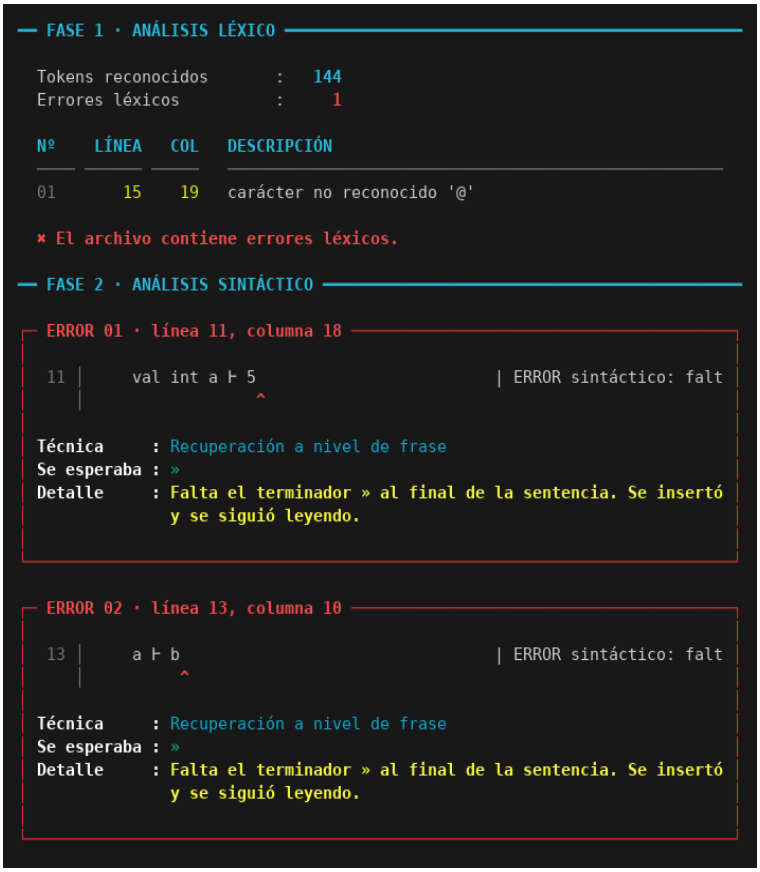
Árbol de ámbitos de pruebasX.cmm.

Prueba 6. Errores léxicos y errores que el programa arregla por frase

Se ejecuta el archivo prueba_recuperacion.cmm, que contiene errores intencionales. En el análisis léxico se detecta el carácter @, se muestra su ubicación y se continúa con la lectura. Luego, en el análisis sintáctico, se encuentran errores por falta del símbolo » al final de una instrucción. El programa identifica dónde falta, lo agrega automáticamente mediante la recuperación a nivel de frase y continúa el análisis sin perder las demás instrucciones.

Prueba 7. Errores comunes con mensaje exacto

El mismo archivo contiene cuatro errores que se reconocen mediante la técnica de producción de error: una variable declarada sin tipo, un arreglo de tipo bool, una instrucción read sin paréntesis y un parámetro sin tipo. En cada caso, el programa muestra un mensaje indicando cuál es el problema y continúa con el análisis sin generar errores adicionales.

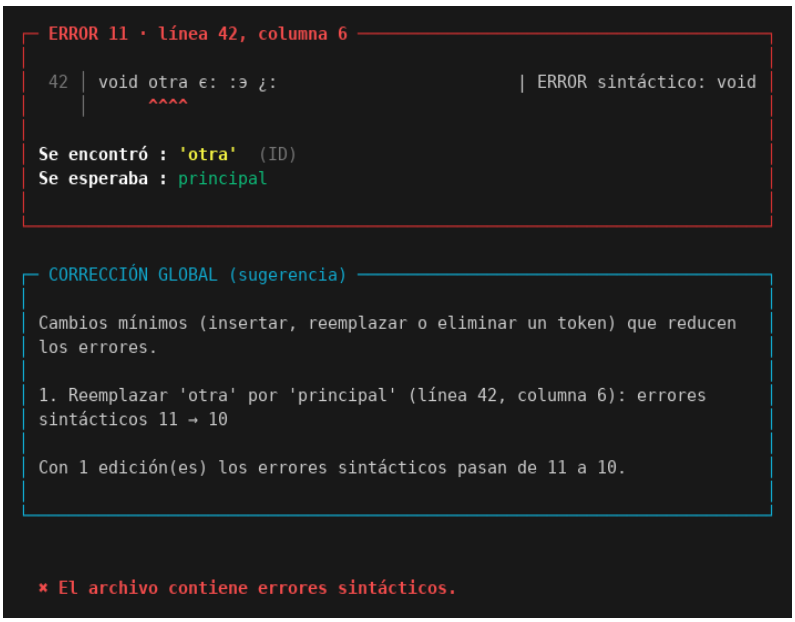


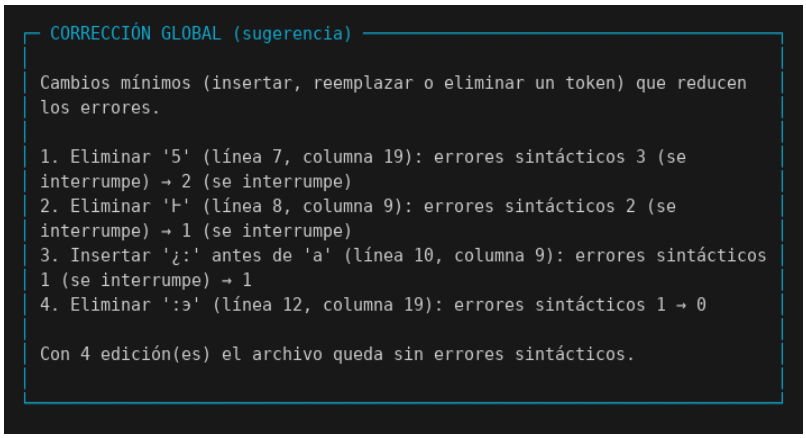


Prueba 8. Modo pánico y corrección global

Cuando el programa no puede arreglar un error de forma exacta, usa el modo pánico: salta hacia adelante hasta un punto seguro y sigue. En la primera imagen la función void otra no es válida, porque void solo se permite en principal. Ese error no tiene una técnica de recuperación específica y lo resuelve el modo pánico. Además, aparece el recuadro Corrección global, que sugiere una forma de arreglar el archivo; en este caso propone cambiar el nombre otra por principal.

En la segunda imagen, el archivo prueba_correccion_global.cmm tiene varios errores juntos: un número de más, un símbolo de más, un bloque sin abrir y un paréntesis de más. La corrección global probó pequeños cambios (quitar, agregar o reemplazar una pieza), volvió a analizar todo el archivo después de cada cambio y encontró que con 4 ediciones el archivo queda sin errores sintácticos.





Prueba 9. Errores semánticos

Se analiza el archivo ejemplo2.cmm, que es un ejemplo del profesor. La fase semántica encontró 24 errores, y cada uno aparece en una sola línea con su número, la línea y la columna del archivo, el tipo de validación que falló y la explicación. Por ejemplo: un nombre repetido, una variable que no fue declarada,

un operador lógico con datos que no son bool, una variable de tipo char a la que se le da un valor decimal y una función que debe retornar un tipo y retorna otro. Al final hay un resumen con la cantidad de errores de cada tipo de validación. Con el otro ejemplo del profesor, ejemplo1.cmm, el programa encuentra 35 errores semánticos.

```
── FASE 3 · ANÁLISIS SEMÁNTICO ──────────────────────────────────────────────

  Ámbitos                :    13
  Símbolos declarados    :    11
  Errores semánticos     :    24

  Nº    LÍNEA   COL  VALIDACIÓN      DESCRIPCIÓN
  ──────────────────────────────────────────────────────────────────────────
  01        5    10  Nombres         '_otra_' ya fue declarado en este ámbito (línea 4)
  02        9    18  Lógica          el operador lógico 'and' requiere operandos bool y recibió 'int' y 'bool'
  03       11    13  Nombres         '_var_' no ha sido declarado
  04       11     4  Nombres         '_var_' no ha sido declarado
  05       15    12  Nombres         '_var_' no ha sido declarado
  06       22    39  Nombres         '_var_' no ha sido declarado
  07       22    55  Nombres         'a' no ha sido declarado
  08       22    57  Nombres         'b' no ha sido declarado
  09       24    26  Nombres         '_var_' no ha sido declarado
  10       26    31  Nombres         '_var_' no ha sido declarado
  11       26    47  Nombres         'a' no ha sido declarado
  12       26    49  Nombres         'b' no ha sido declarado
  13       28    26  Nombres         '_var_' no ha sido declarado
  14       35     5  Asignación      la variable '_otra_' es de tipo 'char' y se le da un valor de tipo 'float'
  15       40     2  Return          la función '_mi_' debe retornar un valor de tipo float, pero se recibió int
  16       44     2  Return          la función '_miOtraFun_' debe retornar un valor de tipo bool, pero se recibió float
  17       47    12  Nombres         '_i_' no ha sido declarado
  18       47     2  Nombres         el arreglo '_arr_' no ha sido declarado
  19       47    27  Aritmética      no se pueden mezclar 'float' y 'int' con el operador '+' (tipado fuerte)
  20       50    10  Entrada/Salida  read solo permite valores de tipo int, float o char; se recibió string
  21       52    11  Entrada/Salida  write no permite valores de tipo bool
  22       53    11  Entrada/Salida  write no permite valores de tipo bool
  23       56    11  Nombres         la función '_f_' no ha sido declarada
  24       56     2  Return          el procedimiento 'principal' no puede retornar un valor
  ──────────────────────────────────────────────────────────────────────────
  Por validación: Nombres 15   Lógica 1   Asignación 1   Return 3   Aritmética 1   Entrada/Salida 3

  ✖ El archivo contiene errores semánticos.
```

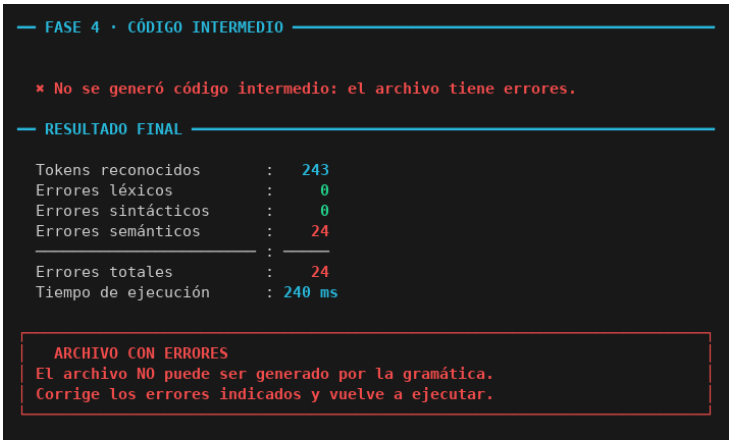
Errores semánticos de ejemplo2.cmm.

Prueba 10. Con errores no se genera código intermedio

En la misma ejecución, la fase 4 indica que no se generó código intermedio porque el archivo contiene errores. Al finalizar, el programa muestra el mensaje El archivo NO puede ser generado por la gramática, junto con un resumen de la cantidad de errores encontrados en cada fase. Como no hay código, tampoco pregunta si se quiere ver.

Prueba 11. Programa sin la función principal

Se analiza el archivo prueba_sin_principal.cmm, que solo tiene una función llamada suma. El programa avisa con el error el programa debe tener una función principal, porque todo programa necesita exactamente una función principal, y no genera código intermedio.



Fase 4 y resultado final de ejemplo2.cmm.



Programa sin la función principal.

Descripción del problema

Muchos dispositivos electrónicos utilizan programas para funcionar. Sin embargo, algunos tienen poca memoria y capacidad de procesamiento, por lo que necesitan instrucciones sencillas y fáciles de procesar.

Por esta razón, en el curso se desarrolló un lenguaje de programación propio que permite declarar variables, realizar operaciones, utilizar funciones, condiciones, ciclos, arreglos y leer y escribir información.

Para que la computadora pueda entender este lenguaje, se necesita un compilador. Este lee el código y lo transforma en instrucciones más sencillas mediante varias etapas:

- Análisis léxico: identifica las palabras y símbolos.
- Análisis sintáctico: revisa que las instrucciones estén bien escritas.
- Análisis semántico: comprueba que las instrucciones tengan sentido.

- Generación de código intermedio: transforma el programa en instrucciones más sencillas.

En el primer proyecto se desarrollaron el análisis léxico y sintáctico. En este segundo proyecto se corrigieron esas etapas y se agregaron el análisis semántico y la generación de código intermedio.

El análisis semántico permite detectar errores que no se identifican únicamente revisando la estructura del código. Por ejemplo, intentar guardar un número decimal en una variable de tipo entero. El lenguaje utiliza tipado explícito y fuerte, por lo que cada variable debe indicar su tipo y respetar las operaciones permitidas.

Entre las principales validaciones se encuentran:

- Comprobar que las variables y funciones estén declaradas antes de utilizarse.
- Evitar nombres repetidos dentro de un mismo ámbito.
- Verificar que las operaciones utilicen tipos de datos compatibles.
- Revisar los parámetros y valores de retorno de las funciones.
- Comprobar el tamaño y los límites de los arreglos.
- Verificar que las condiciones de if, elif, while y for sean de tipo bool.
- Comprobar que exista exactamente una función llamada principal.

Para realizar estas validaciones se utiliza una tabla de símbolos, donde se guarda información de las variables y funciones, como su nombre, tipo, tamaño, ubicación y ámbito.

Además, el programa identifica errores léxicos, sintácticos y semánticos, indicando su línea y columna. También utiliza técnicas de recuperación, como el modo pánico, la recuperación a nivel de frase, las producciones de error y la corrección global, para continuar el análisis cuando sea posible.

Por último, se genera el código intermedio de tres direcciones, donde cada instrucción realiza una operación sencilla.

Diseño del programa

El programa se divide en cuatro etapas: análisis léxico, sintáctico, semántico y generación de código intermedio. Todas se ejecutan en un mismo proceso y al final muestran un resumen. Cada archivo tiene una función: Parser.cup valida las instrucciones, TablaSimbolos.java almacena y busca información, Generador.java genera el código intermedio y Main.java coordina el proceso.

Diagramas

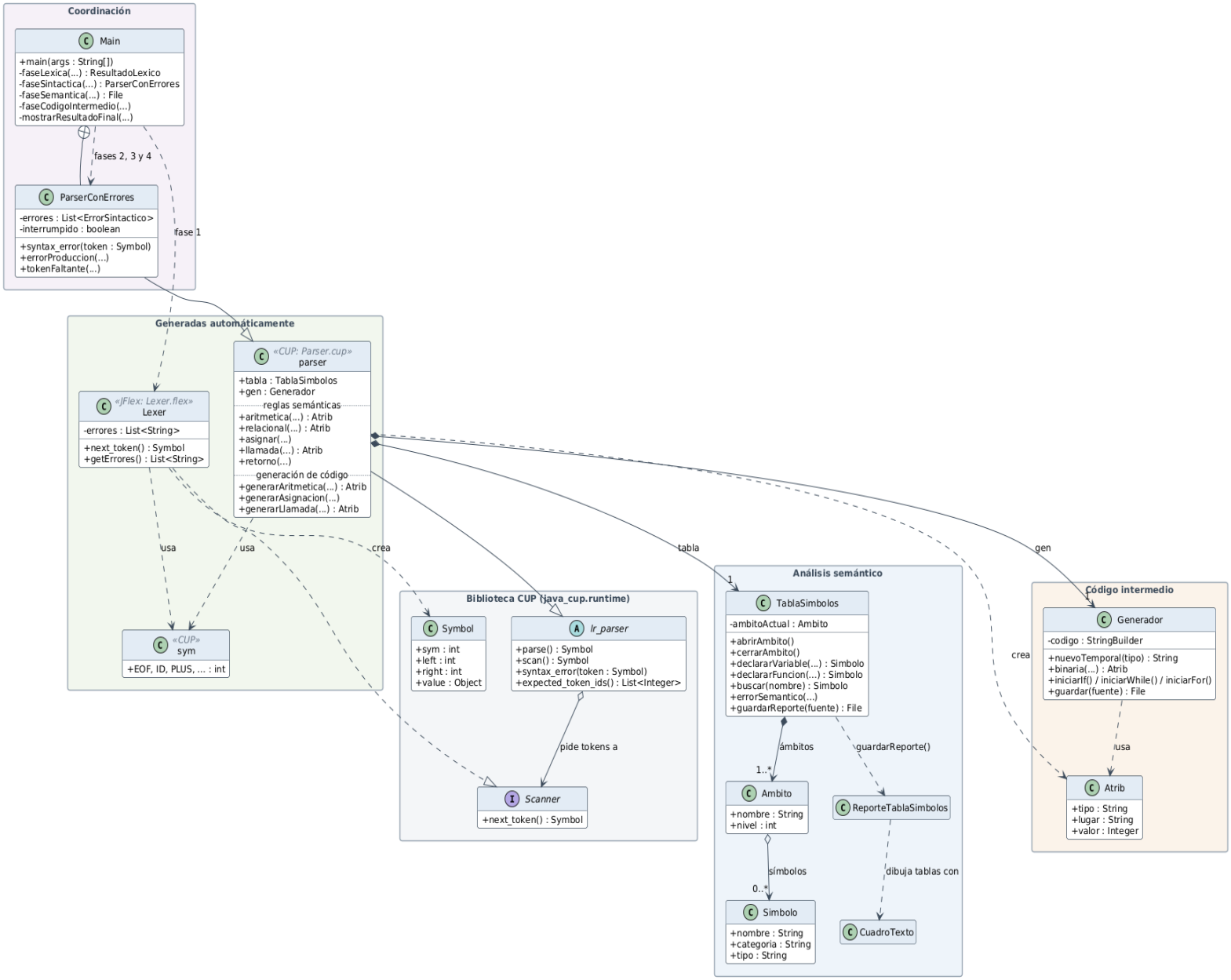


Diagrama de clases general del compilador (generada con inteligencia artificial)

Decisiones de diseño

- Las revisiones de sentido se hacen dentro de Parser.cup, mientras el programa se lee, y no en una etapa separada. Así se revisa cada parte justo cuando se reconoce.
- Cada error semántico lleva su tipo de validación (Nombres, Aritmética, Arreglos…), el mismo que se usa en la sección de reglas de este documento.
- Cada error semántico se muestra en una sola línea, con su línea y su columna, para que sea fácil de leer y de buscar en el archivo.
- Un error que ya se reportó no se repite. Si una variable ya tiene un error, las operaciones que la usan no producen más errores.
- La función principal puede ir en cualquier lugar del archivo, pero debe existir una sola.
- El código intermedio solo se genera cuando no hay errores.
- Los resultados se muestran con colores y recuadros para que cada fase se distinga fácilmente.

Algoritmos usados

Análisis léxico

El analizador avanza por el archivo, carácter por carácter, y busca qué regla del lenguaje coincide con lo que va leyendo. Cuando una regla coincide, crea un token con su tipo, su texto, su línea y su columna. Los espacios y los comentarios se ignoran. Si aparece algo que no coincide con ninguna regla, se registra un error léxico y se sigue con el carácter siguiente.

Análisis sintáctico

Java CUP convierte la gramática en una tabla de decisiones. El analizador lee los tokens de uno en uno y, con la tabla, decide qué regla del lenguaje se está cumpliendo.

Revisión de tipos

Cada valor lleva consigo su tipo. Cuando el programa reconoce una operación, mira el tipo de los dos lados. Si son compatibles, crea un valor nuevo con el tipo del resultado. Si no, reporta el error y marca el resultado con el tipo especial error, para que las operaciones siguientes no repitan el aviso. También se calcula el valor de las constantes enteras (por ejemplo 1 + 1 vale 2), lo que permite saber si la posición de un arreglo existe o si su tamaño es válido.

Tabla de símbolos con ámbitos

Cada vez que aparece el símbolo ¿: se abre un ámbito nuevo, y al llegar a :? se cierra. Los ámbitos forman un árbol: el global es el más grande y, dentro de él, están los de cada función y los de cada if, while o for. Para buscar un nombre, el programa empieza en el ámbito actual y, si no lo encuentra, sube al ámbito de afuera, hasta llegar al global. Si no aparece en ninguno, el nombre no fue declarado.

Al declarar uno nuevo, solo se revisa el ámbito actual, por lo que el mismo nombre se puede usar en bloques distintos. A cada variable se le asigna su tamaño en bytes y una dirección (global+0, pila+4…), sumando el tamaño de las variables anteriores. El global y cada función reparten sus propias direcciones; los bloques de adentro usan las de su función.

Recuperación de errores

El programa usa cuatro técnicas, de la más general a la más específica:

- **Modo pánico:** cuando hay un error, descarta tokens hasta llegar a un punto seguro, como el final de una instrucción, y continúa desde ahí.
- **Recuperación a nivel de frase:** cuando falta un símbolo pequeño (», :ɔ o :?), el programa lo agrega por su cuenta, avisa exactamente qué faltaba y sigue sin descartar nada. Solo se usa donde no hay duda de qué falta.
- **Producción de errores:** la gramática incluye reglas para errores comunes, como declarar una variable sin tipo. Así el mensaje es exacto y no se pierde el resto del análisis.

- **Corrección global:** cuando quedan errores que el modo pánico tuvo que resolver, el programa busca el menor número de cambios que dejan el archivo sin errores. Para cada error prueba quitar una pieza, agregar una pieza o cambiarla por otra, y después de cada prueba vuelve a analizar todo el archivo. Se queda con el cambio que elimina más errores y repite el proceso hasta seis veces. El resultado se muestra como una sugerencia: el archivo no se modifica.

Generación de código intermedio

El código se escribe mientras el archivo se analiza, en el mismo orden en que el programa va terminando de reconocer cada parte. Estas son las ideas principales:

- **Temporales y lugar:** cada número, texto o variable que se usa en una operación se copia a un temporal, y el resultado de cada operación se guarda en otro. El nombre del temporal lleva su tipo (int1, float2, bool3). Cada valor recuerda dónde está su resultado, y así la operación siguiente sabe qué usar.
- **Etiquetas y saltos:** los if, while y for se convierten en etiquetas y saltos (if … goto y goto). Cada etiqueta se arma con el nombre de la función, la estructura, un número y la parte, separados por $: principal$for9condicion. Como un if puede tener otro dentro, el generador usa una pila donde guarda los datos de cada estructura abierta, y siempre trabaja con la última que se agregó.
- **Declaraciones antes del valor:** el parser pide la declaración al final de la regla, cuando el código del valor ya se escribió. Por eso, al leer el ⊢ se marca la posición, y la línea declaracionPrevia# se inserta en esa marca.
- **El incremento del for:** en el código fuente aparece antes del cuerpo, pero debe hacerse después de cada vuelta. El generador lo escribe cuando lo lee, lo recorta y lo vuelve a poner debajo de su etiqueta $inc.
- **Break:** salta a la etiqueta $final del ciclo más cercano.
- **Funciones:** cada función empieza con su etiqueta tipo$nombre y termina con tipo$nombre$final. Una llamada se escribe con un param por cada dato

y un call a la misma etiqueta. Antes de la primera función se escribe goto void$principal, para que el programa empiece en la función principal.

- **Arreglos:** un elemento se escribe como a[i]. Si se usa dentro de una operación, primero se copia a un temporal; si se usa como destino de una asignación, se deja como está.

Automatización

El archivo build.bat busca las librerías, ejecuta Java CUP y JFlex con los archivos de la carpeta gramatica y compila todos los archivos de Java. El archivo run.bat muestra las pruebas disponibles, pide el nombre de un archivo .cmm, lo busca en las dos carpetas de pruebas y ejecuta el programa. Con esto el usuario no tiene que escribir cada comando, lo que hace el proceso más sencillo y con menos posibilidad de equivocarse.

Librerías usadas

El proyecto utiliza dos herramientas externas: JFlex 1.9.1, para generar el analizador léxico, y Java CUP 11b, para generar el analizador sintáctico y realizar las revisiones semánticas. Ambas vienen incluidas en el repositorio. También se utilizan librerías propias de Java para:

- Manejo de archivos: leer el código fuente y guardar los resultados.
- Manejo de texto: mostrar mensajes, errores y generar reportes.
- Colecciones: almacenar tokens, errores, variables y funciones.
- Validaciones: buscar patrones y revisar los mensajes de error.
- Fechas y horas: registrar cuándo se generan los reportes.
- Entrada de datos: recibir las opciones que selecciona el usuario.
- Java CUP: manejar los tokens y la comunicación entre los analizadores.
- Java JDK 21.0.12 para compilar y ejecutar todo el proyecto.

  Los diagramas de la documentación se hicieron con PlantUML.

Análisis de resultados

Objetivos alcanzados

- Se completaron las cuatro fases del compilador: análisis léxico, sintáctico, semántico y generación de código intermedio.
- Se corrigieron y mejoraron los análisis léxico y sintáctico del primer proyecto.
- Se implementó el análisis semántico con tipado explícito y fuerte, con 52 reglas documentadas.
- Se construyó una tabla de símbolos con ámbitos anidados, que se muestra en consola y se guarda en un archivo con el mismo formato.
- Se comprobaron los resultados con los dos ejemplos del profesor, identificando los 35 y 24 errores semánticos correspondientes.
- Se implementó el manejo y la recuperación de errores léxicos, sintácticos y semánticos.
- Se incorporaron las técnicas de recuperación en modo pánico, a nivel de frase, producción de errores y corrección global.
- Se agregó un resumen final con el resultado de cada fase y la cantidad de errores encontrados.
- Se implementó la generación de código intermedio de tres direcciones, con temporales que indican su tipo y etiquetas que no pueden confundirse con variables.
- Se automatizaron la compilación y la ejecución mediante los archivos build.bat y run.bat.
- Se organizó el proyecto en carpetas que separan la gramática, el código, las pruebas y los resultados.
- Se realizaron pruebas para comprobar el funcionamiento de cada parte y evitar errores después de los cambios.

Objetivos no alcanzados

- **Manejo de clases:** el enunciado da puntos extra por incluir clases en el lenguaje, pero pide que funcionen en las cuatro fases. No se implementó.

Razones por las que no se alcanzaron

No se implementaron las clases porque el enunciado no explica cómo deben declararse ni utilizarse. Además, habría sido necesario modificar varias partes del compilador. Se decidió dar prioridad a completar los requisitos obligatorios.

Justificación de toma de decisiones

Por qué la revisión semántica se hizo dentro de Parser.cup

El análisis semántico se implementó en Parser.cup, donde cada revisión tiene su propio método y se ejecuta desde la regla correspondiente. La tabla de símbolos se encarga de guardar y buscar información, mientras que Parser.cup detecta los errores. Así todas las reglas quedan en un solo lugar.

Cómo se guarda la información de cada valor

Cada valor se representa con un objeto de la clase Atrib, que guarda su tipo, el lugar donde se encuentra y su valor cuando es una constante entera. Estos datos se utilizan para las revisiones y para la generación de código intermedio.

Cómo se hace la validación de tipos

Cada operación llama a un método que comprueba si los tipos de datos son correctos. Primero revisa si ya existe un error y luego verifica si la operación está permitida. Las reglas completas, con ejemplos, están en la sección Reglas semánticas del analizador.

Por qué se usó el tipo error

Se utilizó el tipo error para marcar los valores que ya tienen algún problema. Así, las siguientes operaciones reconocen ese error y evitan mostrar avisos repetidos o falsos. Esto facilita identificar el error original sin generar confusiones.

La tabla de símbolos y los ámbitos

Cada bloque tiene su propio ámbito, donde se guardan las variables y funciones declaradas. Esto permite repetir nombres en bloques diferentes. La búsqueda comienza en el ámbito actual y continúa hacia los superiores. Cada símbolo guarda su nombre, tipo, categoría, tamaño, dirección en memoria y ubicación donde fue declarado. Los ámbitos se organizan como un árbol. Además, las funciones se registran antes de analizar su contenido para permitir que puedan llamarse a sí mismas.

Las palabras reservadas como if, while, for o return no se guardan en la tabla, porque no tienen tipo, tamaño ni dirección: las reconoce el analizador léxico. Los if, while y for sí aparecen como ámbitos, porque abren un bloque nuevo.

Decisiones sobre las reglas del lenguaje

- **write:** acepta int, float y string.
- **Condiciones:** los if deben utilizar valores bool.
- **Comparaciones:** los corchetes son opcionales.
- **Función principal:** puede aparecer en cualquier lugar, pero debe existir una sola. Es void, así que no necesita return; el generador agrega el return al final.
- **print:** se eliminó porque no forma parte de las instrucciones solicitadas; se utiliza write.

Decisiones sobre la recuperación de errores

Se utilizó el modo pánico por ser obligatorio. También se agregaron la recuperación a nivel de frase y la producción de errores para casos específicos. La corrección global sugiere cambios, pero no modifica el archivo automáticamente.

Decisiones sobre el código intermedio

- El código intermedio se genera en Generador.java para mantenerlo separado de la gramática.
- El código solo se genera cuando no existen errores.

- Cada instrucción utiliza como máximo tres nombres, como lo solicita el enunciado. Las operaciones largas se dividen en pasos utilizando variables temporales.
- **Las etiquetas usan $ como separador.** Los identificadores del lenguaje pueden tener _ (como _funcionInt_), así que una etiqueta armada con _ podía confundirse con una variable. El analizador léxico no acepta $ en ningún nombre, por lo que una etiqueta con $ nunca choca con una variable. Además, cada parte de la etiqueta es una constante y la etiqueta de una función se arma en un solo método, así un salto nunca puede apuntar a una etiqueta que no existe.
- **Los temporales llevan su tipo en el nombre** (int1, bool2), para saber qué tipo de dato guarda cada uno sin buscarlo.
- **Los números, textos y variables se copian a un temporal antes de usarse**, siguiendo el formato del profesor. Los tamaños y las posiciones de los arreglos se dejan directos (edades[2]) para que el código sea más fácil de leer.
- Los operadores λ y θ siempre evalúan sus dos lados.
- No se realizan optimizaciones, para facilitar la relación entre el código generado y el programa original.

Reglas generales que valen para todas

1. **Tipado explícito y fuerte:** toda variable se declara con su tipo y los tipos no se mezclan (un int y un float no se combinan).
2. **Un error no se repite:** si un valor ya tiene un error, las operaciones que lo usan no producen más avisos.
3. **El análisis continúa:** después de un error el programa sigue revisando y reporta todos los demás.
4. **Declarar antes de usar:** las variables y las funciones deben escribirse antes de usarlas. El mismo nombre sí puede repetirse en bloques distintos.
5. **Con errores no se genera código intermedio.**

Reglas semanticas

1. Nombre repetido en el mismo bloque.

No se permite declarar dos variables con el mismo nombre dentro del mismo ámbito. Sí se puede utilizar el mismo nombre en ámbitos diferentes.

2. Variable no declarada.

Toda variable debe declararse antes de utilizarse, indicando su tipo.

val int a Ⱶ 0» seguido de a Ⱶ 5».

3. Función no declarada o que no es una función.

Las funciones deben declararse antes de llamarse y utilizarse con su nombre correcto.

4. Arreglo no declarado.

Los arreglos deben declararse antes de utilizarlos, indicando su tipo y tamaño.

val int notasʃ:3:ɻ» seguido de notasʃ:0:ɻ Ⱶ 5».

5. Se usa como arreglo algo que no lo es.

Una variable normal no puede utilizarse como arreglo ni acceder a posiciones.

val int xʃ:3:ɻ» seguido de xʃ:0:ɻ Ⱶ 2».

6. Una función se usa como si fuera una variable.

Las funciones deben llamarse utilizando sus paréntesis y argumentos, no como variables normales.

val int r Ⱶ suma ɛ: 1, 2 :ə».

7. Un arreglo se usa sin su posición.

Para utilizar un arreglo se debe indicar la posición del elemento que se desea consultar.

val int x Ⱶ notasʃ:0:ɻ».

2. Aritmética

8. Un operando con error no repite el aviso

Si una variable ya tiene un error reportado, las operaciones que la utilizan no deben generar errores adicionales por el mismo problema.

9. Operandos que no son números.

Las operaciones matemáticas deben utilizar valores de tipo int o float. No se permite realizar operaciones matemáticas con textos.

val int r ⊢ 2 + 1».

10. Mezclar int y float.

No se permite combinar valores int y float en una misma operación, debido al tipado fuerte del lenguaje.

val float r ⊢ 1.0 + 2.5».

11. División entera con decimales.

El operador // solamente permite utilizar valores de tipo int.

val int r ⊢ 5 // 2».

12. División decimal con enteros.

El operador / requiere valores de tipo float.

val float r ⊢ 4.0 / 2.0».

13. Las constantes enteras se calculan (apoyo).

El programa puede calcular operaciones con números constantes para comprobar otras reglas, como los límites de los arreglos.

val int notas∫:1 + 1:ʅ», porque el tamaño calculado es 2.

14. Potencia con exponente negativo.

El operador pot requiere un exponente positivo.

val int r ⊢ 2 pot 3».

3. Comparación

15. Igualdad entre tipos distintos.

El operador == debe comparar valores del mismo tipo.

val bool b ⊢ ∫:1:ʅ == ∫:1:ʅ».

16. Orden entre tipos distintos.

Los operadores <, >, <= y >= deben comparar valores del mismo tipo.

val bool b Ⱶ ʃ:1:ɻ < ʃ:2:ɻ».

17. Comparar textos.

No se permite comparar valores de tipo string. Se pueden comparar caracteres de tipo char.

val bool b Ⱶ ʃ:'a':ɻ == ʃ:'b':ɻ».

18. Una comparación siempre da bool (apoyo).

El resultado de una comparación siempre es de tipo bool y debe guardarse en una variable de ese mismo tipo.

val int a Ⱶ 5» seguido de val bool x Ⱶ ʃ:a:ɻ < ʃ:10:ɻ».

4. Lógica

19. Operador lógico sin valores bool.

Los operadores lógicos ʌ (y) y θ (o) requieren valores de tipo bool.

val bool b Ⱶ ϵ: true :ə ʌ ϵ: false :ə».

20. Negación sin valor bool.

El operador Σ se utiliza para negar condiciones. La gramática no permite escribir una negación con un valor que no sea una condición.

val int a Ⱶ 1» seguido de val bool b Ⱶ Σ ϵ: ʃ:a:ɻ < ʃ:2:ɻ :ə».

21. Una operación lógica siempre da bool (apoyo).

El resultado de una operación lógica siempre es de tipo bool y debe guardarse en una variable de ese tipo.

5. Asignación

22. Valor inicial de otro tipo.

El valor inicial debe coincidir con el tipo de dato declarado para la variable.

val float m Ⱶ 2.5».

23. Asignar un valor de otro tipo.

Una variable solamente puede recibir valores del mismo tipo con el que fue declarada.

val int m ⊢ 1» seguido de m ⊢ 2».

24. Un valor con error no repite el aviso (apoyo).

Si un valor ya tiene un error reportado, no se debe generar otro error por utilizarlo en una asignación.

6. Arreglos

25. Tamaño que no es entero.

El tamaño de un arreglo debe ser un valor de tipo int.

val int notas∫:3:ʇ».

26. Tamaño cero.

Los arreglos deben tener un tamaño mayor que cero.

27. Valores iniciales de otro tipo.

Los valores iniciales de un arreglo deben coincidir con el tipo declarado.

val int notas∫:3:ʇ ⊢ ¿:1, 2, 3:?».

28. Más valores que el tamaño.

La cantidad de valores iniciales no puede superar el tamaño declarado del arreglo.

val int notas∫:3:ʇ ⊢ ¿:1, 2, 3:?».

29. Posición que no es entera.

Los índices utilizados para acceder a un arreglo deben ser de tipo int.

val int x ⊢ notas∫:1:ʇ».

30. Posición fuera del arreglo.

Los índices deben estar dentro de las posiciones disponibles del arreglo. Si tiene tamaño 3, las posiciones válidas son de 0 a 2.

val int x ⊢ notas∫:2:ʇ».

31. Posición negativa.

Los índices de un arreglo no pueden ser negativos.

32. Guardar en una posición un valor de otro tipo.

El valor asignado a una posición debe coincidir con el tipo declarado para el arreglo.

val int notas∫:3:ʇ» seguido de notas∫:0:ʇ ⊢ 25».

33. El tamaño puede ser una operación con constantes (apoyo).

El tamaño de un arreglo puede calcularse mediante operaciones con números constantes, siempre que el resultado sea un entero positivo.

val int szʃ:1 + 1:ɻ».

7. Funciones

34. Cantidad de datos incorrecta en una llamada.

Al llamar a una función se debe enviar exactamente la cantidad de argumentos que necesita.

35. Tipo de dato incorrecto en una llamada.

Los argumentos enviados a una función deben coincidir con los tipos de sus parámetros.

36. Función que retorna string.

Las funciones no pueden retornar valores de tipo string. Los tipos permitidos son int, float, char y bool.

Una función char que utiliza `return 'a'».

37. Parámetro repetido.

Los parámetros de una función no pueden tener nombres repetidos dentro del mismo ámbito.

int f ϵ: int x, int y :ə.

38. void con un nombre que no es principal (sintaxis).

El tipo void está reservado para la función principal. Las demás funciones deben declarar un tipo de retorno permitido.

int otra ϵ: :ə ¿:< return 1» :?.

39. La llamada tiene el tipo de retorno de la función (apoyo).

El resultado de una función debe guardarse en una variable del mismo tipo que devuelve.

40. Función con tipo sin return.

Las funciones que tienen un tipo de retorno deben incluir al menos una instrucción return.

int f ϵ: :ə ¿:< val int a ⊢ 1» return a» :?.

8. Return

41. La función principal retorna un valor.

La función principal no puede retornar un valor, porque es un procedimiento de tipo void.

return».

42. El return no es del tipo de la función.

El valor retornado debe coincidir con el tipo declarado para la función.

Una función int que utiliza return 2».

43. Un return sin valor se permite (apoyo).

El lenguaje permite utilizar return» sin especificar un valor.

return».

9. Control

44. Condición del if que no es bool.

La condición de un if debe ser de tipo bool. Se pueden utilizar comparaciones o variables booleanas.

if ϵ: ʃ:xːɳ > ʃ:0ːɳ :ə.

45. Condición del while que no es bool.

La condición de un while debe ser de tipo bool.

while ϵ: ʃ:xːɳ > ʃ:0ːɳ :ə.

46. Condición del for que no es bool.

La condición central de un for debe ser de tipo bool.

for ϵ: val int i ⊢ 0 » ʃ:iːɳ < ʃ:3ːɳ » i++ :ə.

47. Tipo incorrecto en la cabecera del for.

El valor inicial de la variable utilizada en un for debe coincidir con el tipo declarado.

for ϵ: val int j ⊢ 0 » ʃ:jːɳ < ʃ:3ːɳ » j++ :ə.

48. break fuera de un ciclo.

La instrucción break solamente puede utilizarse dentro de un ciclo while o for.

while є: true :ə ¿:< break» :?.

10. Incremento

49. ++ o -- con un tipo que no es número.

Los operadores ++ y -- solamente pueden utilizarse con variables de tipo int o float.

val int n ⊢ 1» seguido de n++.

11. Entrada y salida

50. read con un tipo no permitido.

La instrucción read permite leer valores de tipo int, float y char.

read є: n :ə», donde n es de tipo int.

51. write con un tipo no permitido.

La instrucción write no permite mostrar valores de tipo bool.

write є: "hola" :ə».

12. Programa

52. Falta la función principal.

Todo programa debe tener exactamente una función llamada principal, que representa el punto de inicio de la ejecución.

Declarar una función void principal є: :ə ¿:< :?.

Bitácora

La bitácora se genera automáticamente con Git, con el commit de cada integrante y su comentario:

https://github.com/Alice-Arias/PY2-Analisis-Semantico-y-Generacion-Codigo-Intermedio-Compiladores/commits